April 2, 2026

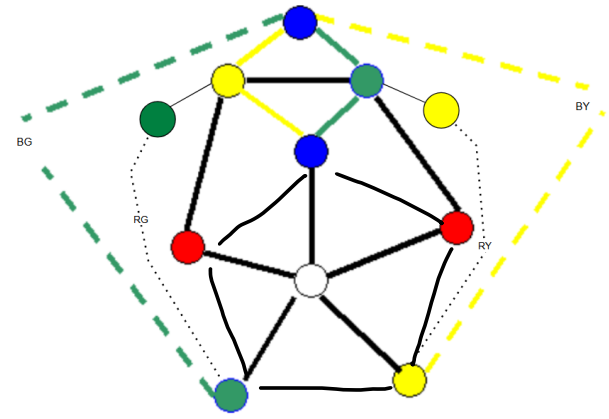
- Kempe 2.0 Animation show and introduce the failing condition.
    - Minimal Length L
    - Key: all neighbours of a deleted node will not be the (n+1)th colour.
- Kempe 1.0 and Kempe 2.0 on the New counterexample with K3,3.
    - Old vs new counterexample.
    - No K5 minor is 5 colourable by Kempe 2.0?
- Kempe 2.0 for no K6 minor is 6- colourable?
    - No kt minor is t-colourable by Kempe 2.0.
    - Kempe 2.0 works for 5-degenerate graphs right now, no K7 minor is 7 – colourable?
    - Exceptions and degeneracy trap.
- Show interesting proof of Mader's theorem using pure graphs.
-

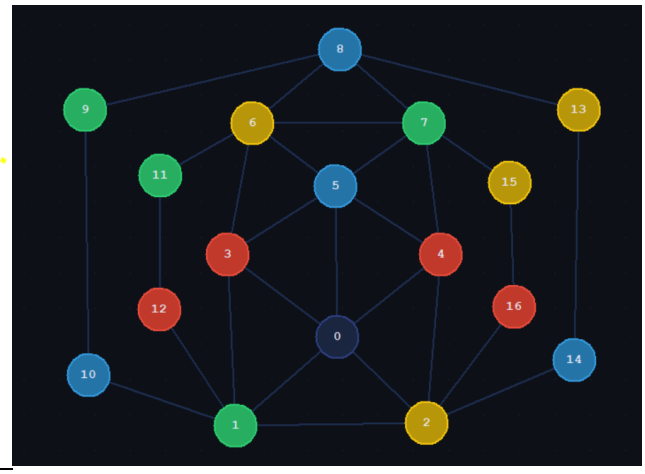
Todo:

- Scan work and cite contributors, resources in check
- Document kempes method cases
- Better explain kempe 1.0 and 2.0 with proof and methodology.
- Link resources like graph visualizer tool and all othe resources.
- A maximally kN graph is a pure graph.
- Wagner and k3,3 thing
- The exceptions are on edge count but not degeneracy right???
- Can link online video

Kempe 1.0:

Kempe's inductive colouring method was proved unable to 4-colour all planar graphs by the following Heawood-inspired counterexample:





Counterexample base structure from Shai Simonson: https://sites.google.com/site/shaisimonson/professional-page/mathexp/lab4.

Counterexample was made precise by Professor Nehaniv, Hanna, Amena, and Alex.

Original Kempe Proof (without counterexample):

Since every planar graph has a node with degree < 6 (5-degenerate), removing a node with degree < 6 forms an induced planar subgraph also possessing a node with degree < 6. So, using Kempe's method for colouring nodes with degree < 6, we can colour all nodes recursively in the reverse order of deletion until the entire graph is coloured.

Proof by Induction:

P(n): every simple, planar graph with n vertices is 4-colorable.

Base case:

P (1) is true since a single node can be coloured with 1 colour, 1 < 4.

Inductive step:

Inductive hypothesis- Assume P(k) is true for some k.

Then add an uncoloured node to the graph to check P(k+1). If the added node has degree < 6, apply Kempe's method and 4 colour the graph.

If the added node has a degree >= 6, remove a coloured node with <6 neighbours, by 5-degeneracy this node must exists, and by the inductive hypothesis P(k), the induced subgraph is 4 colourable. Then add the deleted node back in (as an uncoloured node) and apply Kempe's method to 4-colour the graph.

So, P(k+1) is true.

By mathematical induction, Kempe's method with fact of 5-degeneracy of planar graphs proves the four-colour theorem.

Solving the Counterexample with Kempe-based 5CT:

Although failing to 4 colour planar graphs, Kempe's method proves the 5CT for planar graphs. On the case that the counterexample rises, the to-be-coloured node could be coloured the fifth colour, making Kempe's method complete for 5-colouring all planar graphs.

"Kempe's flawed method never fails to five-color a graph." -Shai Simonson

https://sites.google.com/site/shaisimonson/professional-page/mathexp/lab4

Kempe 1.0 Animation Program:

An **O(n^2)** program that 5-colors any **simple, planar uncoloured graph** using Kempe's method and animates the colouring, removal, and repopulating of nodes.

In our program, we colour the uncoloured node White for the counterexample to Kempe's method.

The **input** to the program should be a simple, planar graph adjacency (symmetric) matrix in plain form, e.g.

```
0 1 1 0 0 0
1 0 0 1 1 0
1 0 0 0 1 0
0 1 0 0 0 1
0 1 1 0 0 1
0 0 0 1 1 0
```

The program requires 5 degeneracy and hence may still work if the graph is non-planar and 5-degenerate, but certainly not for all cases. This method fails for 5-degenerate, non-planar graphs when a to-be coloured node has 5 neighbours of all 5 colours {R, G, B, Y, W} and each neighbour has a Kempe chain with another neighbour. This can be easily created using non-planar structures, e.g. K3,3, K5 minor graphs.

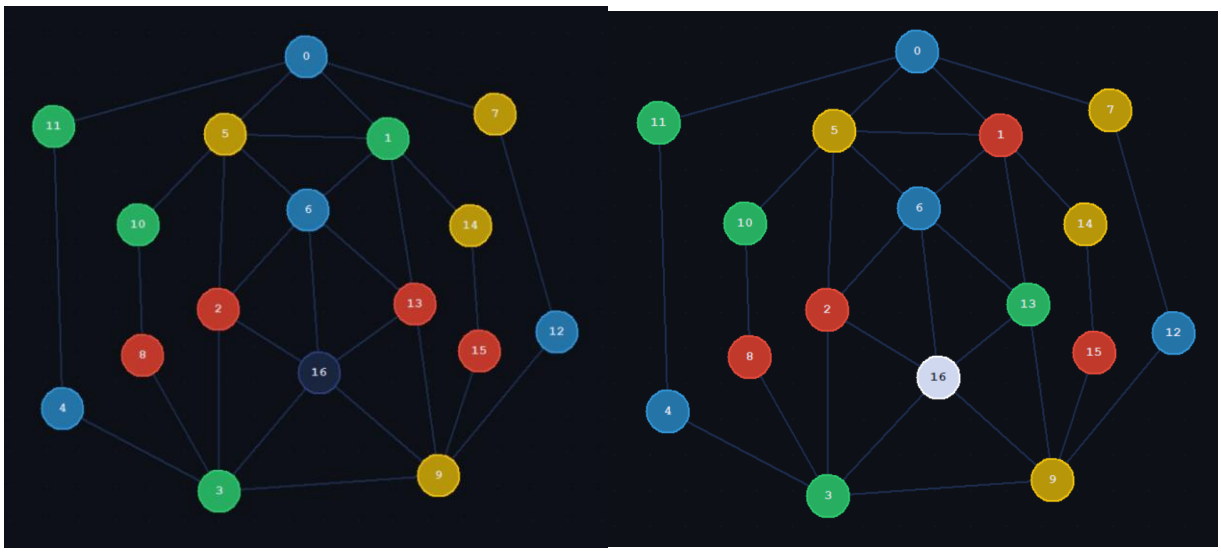
Note: an uncoloured simple, planar graph will be 5-colourable using the Kempe-based 5CT program/animation. An already coloured simple, planar graph (with proper colourings) will also be 5-colourable, but requires manual set up in the code to exhibit. See ... For details on how this can be done.

Animation:

See the animation program and videos under Kempe 1.0/ in the GitHub repo.

Example:

Applying the program to the Kempe counterexample produces the following:



Node 16, for the counterexample case, is coloured the fifth colour.

Kempe 2.0:

Method:

1st problem: Removing Coloured Nodes can/will destroy Kempe chains.

Proof: : Destroyed Kempe chains which the program does not currently realize will get caught at the end and the fifth colour will be used to remove colouring conflict and prove 5CT

A to be deleted node has neighbours of colours in the set = { R, G, B, Y}, the deleted node won't be originally coloured white, due to the nature of the algorithm (returns -2 [kempe-algorithm error code]). Hence, it will not be possible for two adjacent nodes to be coloured White at the end, after the correction, since a deleted node only has {R, G, B, Y} neighbours and the colouring correction only applies to deleted nodes.

The correction makes true that the assumption that some kempe chains may not exist, by recolouring certain nodes white.

Using the same Kempe-method in a different algorithmic approach may be able to colour any 5-degenerate graph. It works on the same case by case basis as the original Kempe 1.0 5CT method above, but instead of deleting all 5 degenerate nodes into a stack and recolouring them (by popping from stack), it first colours all the neighbours of a 5-degenerate node, and then colours the to-be coloured node using the Kempe method, and then deletes this node.

Kempe 2.0 uses the 5th colour when it is restoring nodes that were previously deleted once all its neighbours were properly coloured. Using the 5th colour

removes assumptions that certain Kempe chains didn't exist after removing these nodes. When the deleted node is restored, there may be a colouring conflict, and the 5th colour solves this conflict.

<u>Program Time Complexity:</u>

toggleKempeChainO(V + E) ≈ O(V^2)

KempeSetO(V^2 log V)

colourO(V^2)

colourNeighboursO(V^3)

deleteNodeO(V)

preProcessingO(V^2)

deletedNodeErrorO(V)

main (overall)O(V^4)

Conjecture:

Kempe 2.0 is an approach that 5-colors any **simple graph with no K6 minor** if the input graph follows Property *.

Property *: A graph G with N nodes numbered from node i, j ∈ [0 to N-1] where every node *i* has a neighbour *j* such that <u>| i – j | = 1</u> (Property *). This results in the graph being coloured such that the next to-be-coloured node has the shortest distance L from an already coloured node.

Property * is analogous to colouring the graph in continuous manner.

“”Pre-coloured graphs will be 5-coloured successfully only if it the input graph's colouring could be obtained naturally with the algorithm, and (*) holds as well.

- E.g. the counterexample.””??

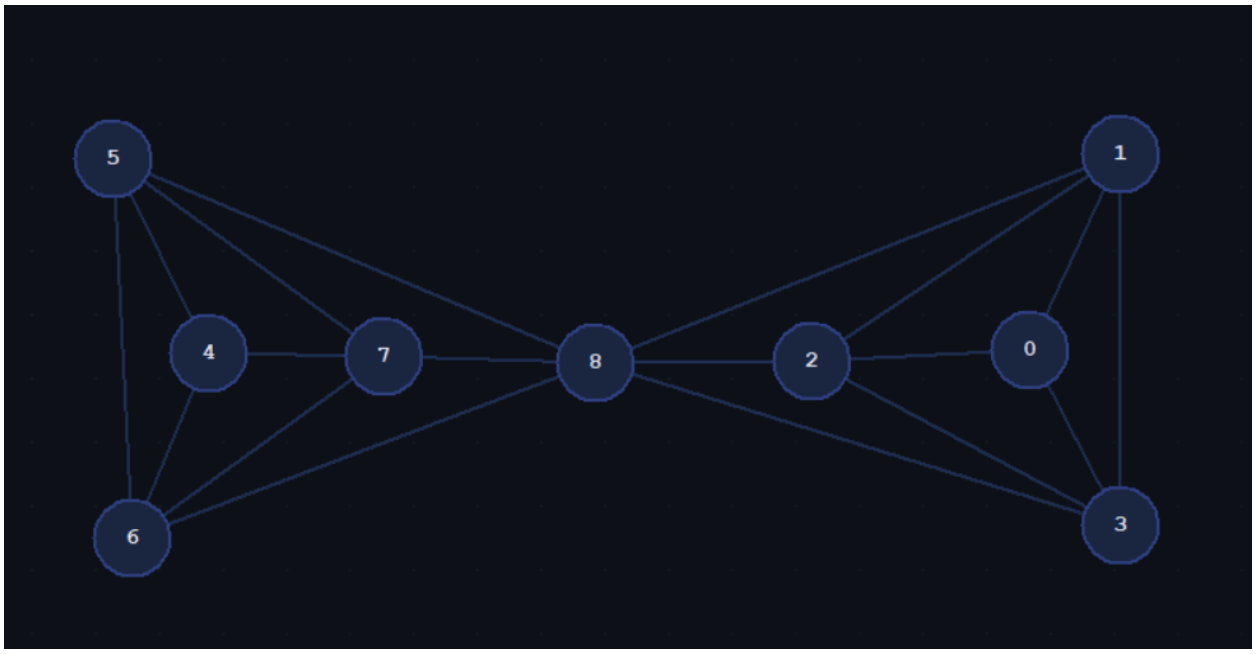
Graphs and Property *:

When Property * is not held, it becomes possible for an uncoloured node to have more than 5 coloured neighbours of the entire colour set {R, G, B, Y}, hence uncolorable, using Kempe's method (as it is not <= 5-degreed).

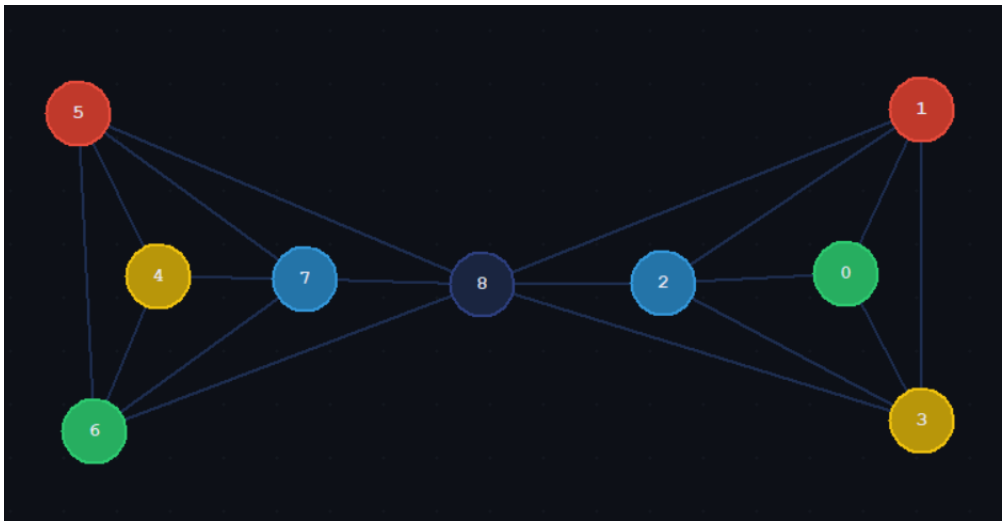
The reason this occurs is because without Property *, it is possible for certain 5-degenerate nodes to be coloured and deleted, resulting in a graph with an uncoloured node with >5-coloured neighbours, where the neighbours' colouring consist of all 4 colours {R,G,B,Y}. This is not colourable using Kempe's method. Without Property *, the graph "deadlocks", like in concurrent programming.

Property * star makes it such that as we apply Kempe 2.0, we colour only adjacent nodes. So, we build the graph colouring incrementally from uncoloured nodes with shortest L from a coloured node. And we may never unintentionally colour neighbours in a random order such that an uncoloured node with >= 6 neighbours of each colour persists.

Property * is not held for the below graph.



Since, property * is not held, colouring in numerical order (which the program does in ascending order), results in a **deadlock** when colouring node 8, the final node. Node 8 would not be seen as 5-degenerate once the program reaches it. It would have 6 coloured neighbours. Hence, nodes must be coloured in a continuous manner, ensuring Property *.



Property * Implementer algorithm:

I considered developing the feature to reorganize the graph such that property * applies, but I decided to not implement this feature as the easy solution is to make sure the inputted graph has property *.

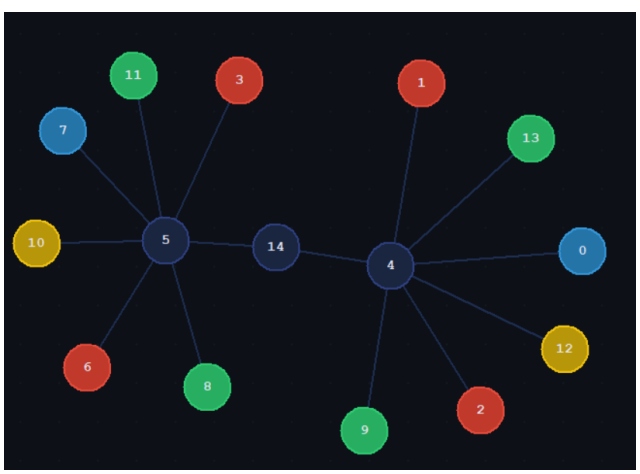
Implementing this feature would be standard but take some time. It would be an algorithm that reorganizes and numbers the nodes of a graph in a continuous order.

Further work is required to investigate the necessary and sufficient conditions of Property * for Kempe 2.0 to work. From previous graph test cases, Property * seems to be a sufficient but not necessary condition.

Pre-coloured graphs:

pre-coloured graphs may very easily result in the deadlock condition. If certain non-adjacent nodes are coloured, it is equivalent to not following Property * when colouring the graph.

E.g.



This graph does not follow Property * since if the some of the neighbours of node 5 are all coloured, node 5 should be coloured next, assuming no other shorter L connection, as seen in graph example.

Conclusion:

An input graph must follow property *. If originally uncoloured, then the program will provide the 5-colouring of the graph. If pre-coloured, it must be coloured such that no uncoloured node has >5 coloured neighbours with entire {R, G, B, Y}, or in other words, it can be obtained naturally using the algorithm, as with the counterexample.

Working cases:

- Uncoloured:
    - Guaranteed will work under property star.
- Coloured:
    - May work under property star
    - Guaranteed will fail if deadlock condition is already existing

Goal:

I'm trying to show that all graphs with no Kt minor are t= colourable using the recursive process. Even though there is a degeneracy trap, it seems that these traps are <t-colourable.

All Kx,x,x,x,x,x,,.... Where the number of xs less than t are t-1 colourable!

Even the "weaker" statement—that a graph with no K_7 minor is 7-colorable—is **currently unproven**.

f you have a proof that K_6-minor-free graphs are **6-colorable** without using the Robertson-Seymour-Thomas "apex reduction," it would be considered a **major theoretical breakthrough** in structural graph theory.

Using Kempe 2.0 for Hadwiger's Conjecture:

In graph theory, the Hadwiger conjecture states that if G is loopless and has no Kt minor then its chromatic number satisfies χ(G) < t.

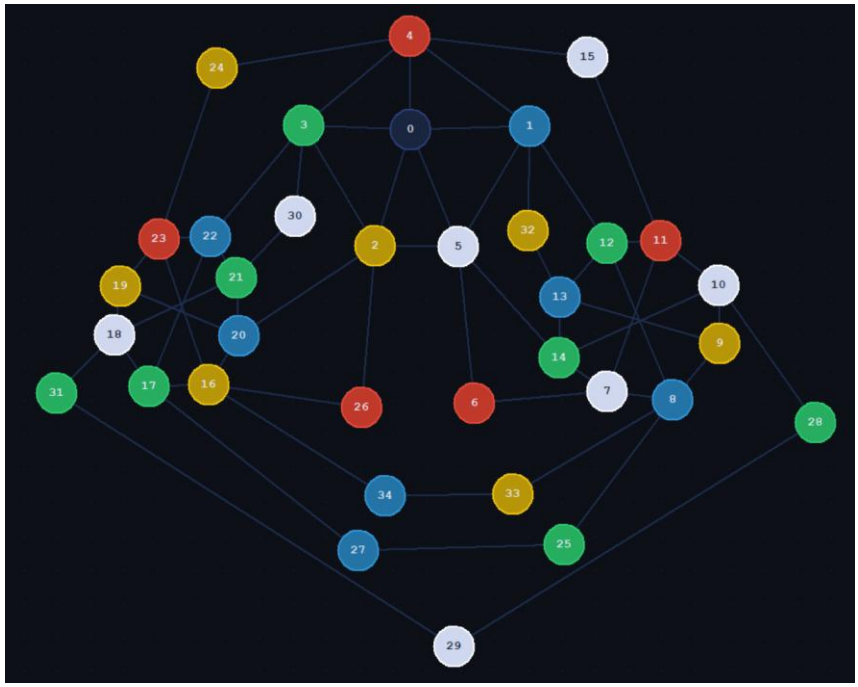
I try to investigate that a graph with no K6 minor is 6 colourable.

AI gemini: 3. The Hadwiger Bottleneck

If there were a "weaker" method to prove $\chi(G) \leq 6$ for K_6-minor-free graphs, it would likely provide a template for the K_7 case (K_7-minor-free $\implies$ 6-colorable), which remains one of the most stubborn open problems in the field.

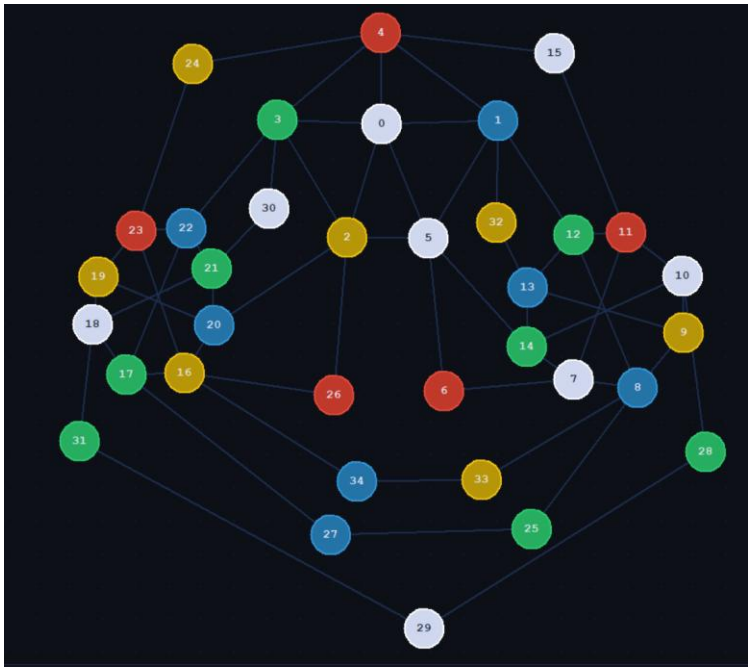
Kempe 1.0 and 2.0:

Let's assume a 5-degenerate graph with no K6 minor, we found the following counterexample for Kempe 1.0 when trying to 5-colur graphs with no K6 minor. Kempe chains between all 5 neighbours of an uncoloured node can occur using K3,3 (W8 used here). K5s probably can be used to form Kempe chains between all neighbours of node 0.



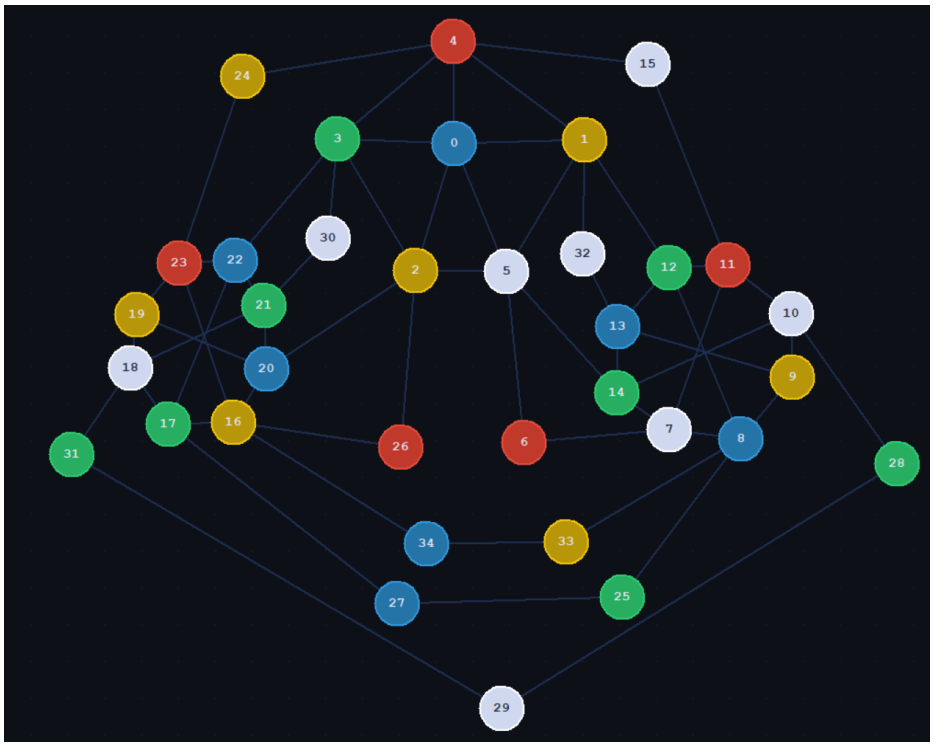
Kempe 1.0:

Kempe 1.0 failed and defaulted to coloring node 0 white, creating a colouring conflict. Hence, Kempe 1.0 is insufficient for HC(6).



Kempe 2.0:

Kempe 2.0 succeeded in 5 colouring the graph.



Note: since both graphs are already coloured, they required manual set up in the programs. See the respective videos for their colouring animations.

Although Kempe 2.0 must follow Property *, we know the counterexample follows it, since the only uncoloured node has only 5 neighbours, hence no deadlock occurred.

Kempe 2.0 for HC(6):

3.1 Theorem: If G is k-degenerate then its chromatic number is at most k +1.

Proof: colour all neighbours then the node which takes <= k+1 colours, for all nodes. Prove inductively.

A regular graph is a graph where every vertex has the same number of neighbors, meaning all vertices have the same degree. Therefore a k-regular graph is k-degenerate and k+1-colourable.

3.2 Theorem: For every graph H there exists c such that |E(G)| ≤ c|V(G)| for every graph G with no H minor.

3.3 Theorem: For $t \leq 7$ and all $n \geq t-2$, every n-vertex graph G with no Kt minor satisfies $|E(G)| \leq (t-2)n-(t-1)(t-2)/2$. CITE Mader.

But for $t \geq 8$ the pattern fails

Proof:

Starting example: We know e <= 3v – 6 for a planar graph. By Kuratowski's theorem, a planar graph has no K3,3 or K5 minor.

K3,3 minors do not change degeneracy of graphs with no K5 minor, since their degeneracy is lower. Proved later.

First example, use a maximally K4 graph:

A K4 graph has (n)(n-1)/2 eges = (4)(3)/2 = 6 edges.

Every added vertex connects to 3 other vertices so that the graph continues to be maximally K4. The number of added vertices is (v-4). Therefore e ← e + 3(v-4)

Therefore e = 3v-12+6 = 3v – 6 for maximally K4 graph. And hence e <= 3v-6 for a graph with no K5 minor. This matches with the fact that e <= 3v-6 for planar graphs. So it seems that K3,3 minors don't affect the max bound on edge count, E(v), only the Kt minor?

The average degree of a graph with an edge bound is 2e/v;

Since e <= 3v-6, average degree of a graph with no K5 minor <= 6-12/v <= 5.

The degeneracy of a graph can be determined by the average degree.

Therefore, the degeneracy of a graph can be determined by graph minors.

The degeneracy is therefore 5 for a graph with no K5 minor.

Generalize:

A graph with no Kt minor has a maximum number of edges if the graph G (with no Kt minor) is maximally Kt-1.

A maximally Kt-1 graph has edges...

E = (t-1)(t-2)/2 + (t-2)(v-(t-1))

= (t-1)(t-2)/2 + (t-2)(v) – (t-2)(t-1)

= - (t-1)(t-2)/2 + (t-2)(v)

E = (t-2)(v) - (t-1)(t-2)/2

A graph with no Kt minor has e <= (t-2) (v) - (t-1)(t-2)/2

Average degree = 2e/v

- ⇨ average degree <=

= 2(t-2) - (t-1) (t-2)/v

=(t-2) (2 – (t-1)/v)

As the number of vertices (nodes) increases the average degree increases.

V != infinity for a finite graph.

To get an upper bound for the average degree, let v approach infinite, but not equal it, for finite graph.

Therefore, average degree < = (t-2) (2 – 0+) = (t-2) 2-

So, max degree of a graph with no Kt minor is < 2(t-2)

Therefore, a graph with no Kt minor is 2(t-2) – 1 = 2t-5 degenerate.

The reverse means that a graph with a Kt minor has an average degree >= 2(t-2).

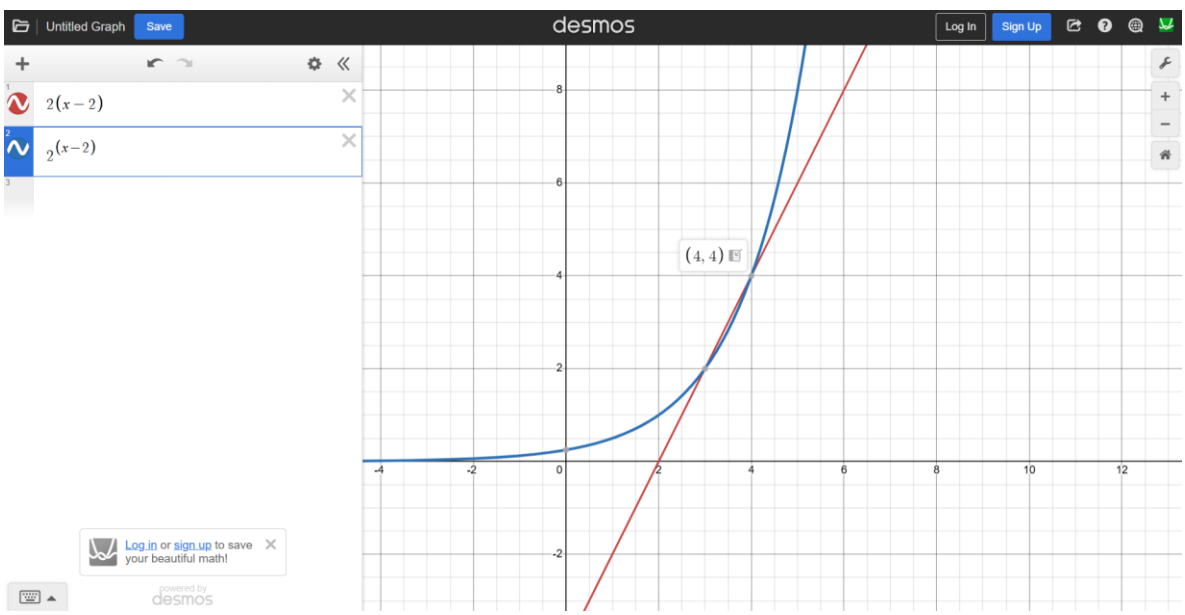
Mader's Thorem:

Mader's theorem states that If the average degree of G (which has n vertices) is at least 2^{t-2} then G has a K_t minor.

- https://math.stackexchange.com/questions/4946734/maders-theorem-large-degree-force-a-minorproof-by-induction

Proof:

a graph with a Kt minor has an average degree >= 2(t-2). 2^(t-2) >= 2(t-2) for all t in N. Can solve algebraically or see desmos plot below.



Degeneracy (G) = max(degen(G1), degen(G2))

Proof:

Defintion of degeneracy: a k edgerante graph is an undirected graph in which every subgraph has at least one vertex of degree at most k.

1. Every subgraph with a node not in the clique sum will have a max degeneracy of max (degen(G1), degen(G2)). This is because since the graphs were originally k1 and k2 degenerate, the clique sum can only increase it to its max.
    a. Assume a node in the clique sum was only k-degenerate node in a subgraph, joining G1 AND G2 on clique sum, may result in. investigate more with k regular graphs.
    b. But it's average so the average will be in the middle, intuitive understanding. But not so easy.

Because every subgraph contains a node with degree less then equals to K then Assume that the node in the clique sum was the only node in a sub graph, where it was k degenerate. This node is joined to another graph via a clique sum, then that node could be attached to more than the K degrees. But this subgraph is still max(degen(g1), degen(g2)) because

This is based on graph minors theory actually, since both graphs are k1, k2, degenerate, they do not have have a kt,q+1 minors respectively, hence by joining, the result has a maximum of a kmax(t,q) + 1 minor.!!

Joing a clique sum, may join a part of a minor, but CAN NOT increase the minor size of a graph, b definition of a clique usm.

Solved!!!

, but the nodes that are formed with it will not be because every not clique sum node is K 1 or k2 degenerate so that's a graph that we attach onto that note through the click some was is actually K degenerate either K one or K2 degenerate hence the the degeneracy ask Vance K one or a key to Since every sub graph is, Katie degenerate every graph the heavy grass Since every sub graph is Kate 10 minutes, you can click Sam's to glue to grass, G1 and G2 with K-1 OK to degeneracy I need some craft that has an old KO a note note, and that is not an old in the click son will be Casey Jenrette

2. The subgraph with nodes only in clique sume has dengeracy (k-1), since it was joined by a k-clique.

No K6 minor is 5-degenerate TRAPPED:

Maximally K5: degen = 5
Planar: dgen = ;
Wagner(k3,3) = 3

The degeneracy of a graph with no K6 minor seems to be therefore 5. I am not sure if this is true or not based on the lack of this result in literature, but I believe it is true. Maybe, there was no need to find the lowest degeneracy of graphs with no K6 minor, as Kempe's method was obviously not capable, as seen in the counterexample. But our modified-kempe's method may successfully color any graph with no K6 minor.

I FELL INTO THE DEGENERACY TRAP, SEE AMENA'S LECTURE AND LITERATURE ON GRAPHS WITH NO K6 MINOR BEING 6 REGULAR LIKE K2,2,2,2,2, BUT THEY ARE 5 COLOURABLE IT SEEMS OR 6 COLOURABLE LIKE K2,2,2,2,2,1.

2.2 Theorem: The graphs with no K5 minor are precisely the graphs that can be built by repeated 0-, 1-, 2-, and 3-sums, starting from planar graphs and copies of V8.

Proof:

It is proved that a planar graph has no K3,3 or K5 minor. A graph with no K5 minor therefore is built by planar and K3,3. A graph with no K4 minor has K1, K2, K3, K4 minors

V8 is a graph such that any 2 clique sums with a planar graph stil yields k2,2, embedded in v8. V8 can contract and 2 distinct pairs and still have K3,3.

Appendix:

Todo:

- Ask ai to find citations for degeneracy of a graph with no k6 minor, other than Mader's Theorem. Does it exist for 5 (our number)?
- Try to find a counterexample or counterreason to modifed kempe's method

```
// Adjacency matrix + color: int adj[35][36]
// Columns 0..34 = adjacency, column 35 = color
// 0=none 1=red 2=green 3=blue 4=yellow 5=white
// {
{0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
{1, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0},
{1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0},
{1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0},
{1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
{1, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
{0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
{0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
{0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0},
{0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
{0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0},
{0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
```

```
{0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
{0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0},
{0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
{0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
{0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1},
{0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0},
{0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0},
{0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
{0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
{0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0},
{0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
{0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
{0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
{0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0},
{0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
{0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
{0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0},
{0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0},
{0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
{0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0},
{0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
{0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1},
{0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0}
// };
```

```
0 1 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
1 0 0 0 1 1 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0
1 0 0 1 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 0 0
1 0 1 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 1 0 0 0 0
1 1 0 1 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0
1 1 1 0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 1 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 1 0 1 0 0 1 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 1 0 1 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 1 0
0 0 0 0 0 0 0 0 1 0 1 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 1 0 1 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0
0 0 0 0 0 0 0 1 0 0 1 0 1 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 1 0 0 0 0 0 0 1 0 0 1 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 1 0 0 1 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0
0 0 0 0 0 1 0 1 0 0 1 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 1 0 0 1 0 0 1 0 0 0 0 0 0 0 1
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 0 0 0 1 0 0 0 0 1 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 0 1 0 0 0 0 0 0 0 0 1 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0
0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 1 0 1 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0
0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 0 1 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 1 0 0 1 0 1 0 0 0 0 0 0 0 0 0
0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0
```

0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 1 0 0 0
0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0
0 1 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0

// Color column only (35 nodes, 34 colored)
// 0=none 1=red 2=green 3=blue 4=yellow 5=white
// int colors[35] =

Counterexample: 0 3 4 2 1 5 1 5 3 4 5 1 2 3 2 5 4 2 5 4 3 2 3 1 4 2 1 3 2 5 5 2 4 4 3
Modified: 3 4 4 2 1 5 1 5 3 4 5 1 2 3 2 5 4 2 5 4 3 2 3 1 4 2 1 3 2 5 5 2 5 4 3
Original: 5 3 4 2 1 5 1 5 3 4 5 1 2 3 2 5 4 2 5 4 3 2 3 1 4 2 1 3 2 5 5 2 4 4 3

0 1 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0
1 0 1 1 0 0 0 0 0 1 0 1 0 0 0 0
1 1 0 0 1 0 0 0 0 0 0 0 0 1 0 1
1 1 0 0 0 1 1 0 0 0 0 0 0 0 0 0
1 0 1 0 0 1 0 1 0 0 0 0 0 0 0 0
1 0 0 1 1 0 1 1 0 0 0 0 0 0 0 0
0 0 0 1 0 1 0 1 1 0 0 1 0 0 0 0
0 0 0 0 1 1 1 0 1 0 0 0 0 0 1 0
0 0 0 0 0 0 1 1 0 1 0 0 0 1 0 0
0 0 0 0 0 0 0 0 1 0 1 0 0 0 0 0
0 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0
0 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0
0 1 0 0 0 0 0 0 0 0 1 0 0 0 0 0
0 0 0 0 0 0 0 0 1 0 0 0 0 0 1 0
0 0 1 0 0 0 0 0 0 0 0 0 0 1 0 0
0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 1
0 0 1 0 0 0 0 0 0 0 0 0 0 0 1 0

// Color sequence (17 nodes, 16 colored)
// 0=none 1=red 2=green 3=blue 4=yellow 5=white
// int colors[17] =
{0, 2, 4, 1, 1, 3, 4, 2, 3, 2, 3, 2, 1, 4, 3, 4, 1};

Original: 5 2 4 1 2 3 4 1 3 2 3 2 1 4 3 4 1
Modified: 2 3 4 1 1 3 4 2 3 2 5 2 1 4 3 4 1

Kempe 2.0 Notes:

NOTES SECTION:

// What does Kempe's method even serve to achieve?

// A planar graph has a node with <6 vertices.

// What we know:

// There exists a node with less than 6 nieghbours in every graph. Let us act upon it and remove it such that we can find another

// one and guarantee it, by the equation. Everyhting right so far.

// Apply kempes method on it for a node with less than 6 neighbours. It should work since it has less than 6 coloured nieghbours

// There are some cases that don't work, the counterexample, therefore the kempes method doesn't work

// Kempe's emthod works by not delteing nodes and assuming that it it's true that by colouring a node with < 6 neighbours,

// it affects the graph such that a new node with less than 6 uncoloure dniehg bours is formed, should be true,

// Then keep on going, but there are bad cases like the counterexample, by not deleting stuff.

// This is false, since it's not ht enumbber of uncoloured nieghours, but the number of neighbours!!!

// Our 5CT method works by showing that the counterexample can be counteracted, we didn't want to show this by colouring the central

// node by memorizing when this case happens and colouring the centre black (the 5th colour) because

// Big question: so removing seems right, but why do not remove in 4CT way? do we just assume that an error happens due to the

// inevitable kempe chain and call it quits? Yes!!!!

// Now the big question is whether

//we need to choose a node with < 6 niegbours not <6 uncoloure dneighbours, since the nieghbours should b ecoloured, by

// kempes method.

// it seemed weird that the counterxample is already populated.

// but realizing it is not a predetermined colouring grah, but a graph that COULD be obtained

// from the kempe method was critical. SOme graphs simply can't be obtained.

// E.g. 5 neighbours(uncoloured) each neighbour with 6 cooured neighbours. This would brick up the system.

// Realizing that it cant ever be obtained using our method from an originally uncoloured graph was critical

// in me understanding that the roginal graph must be uncoloured. And that the counterexample

// is a version that CAN be ontained and fails. I think it CAN be obtained intuitively.

// It fails using the same method of removal, the failure is due to an inevitable kempe chain. Hence

// always failing, it can be obtained m the removal method.

// this method counteracts it and shows that all planar graphs which contain a node degree with < 6 can be 5 coloured.

// preprocessing is necessary to show that the graph can be obtained and is then solved, by the 5CT.

// original method works except for the last case (very obvious now), and it is critical to be convinced that this case

// can occur using this removal method. things that can not be obtained, can't be solved using 4CT or 5CT method shouldn't be

// considered, hence a n empty graph should be input, only graphs wchich ca n be obtained can be originally colored, but it is

// ad hoc and it should only include the counterexample as the only valid graph that has an input coloring.

// therefore, 5CT rpvoed.

//CRITICAL: ASSUME THE COUNTEREAXAPLE CAN BE OBTAINED NATURALLY USING KEMPE METHOD, WE SHOW THAT IT DOESN'T MATTER, by 5CT.

// we don't even need to show the xounterexample, assuming it can be obtained, we set our algorihtm for 5XT!!